# Object Oriented Programming using C++
# Inheritance

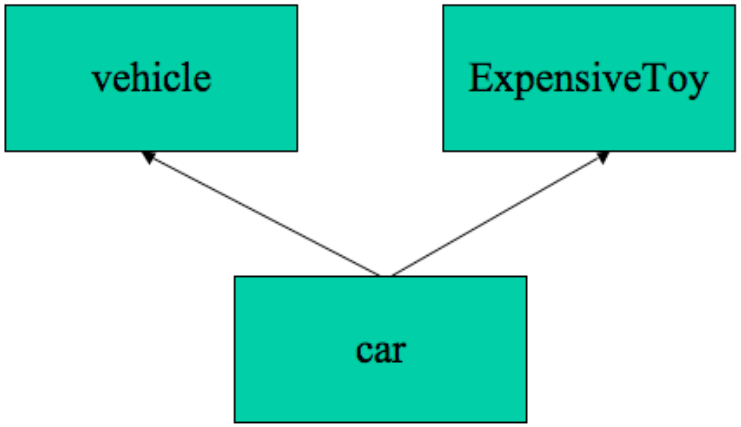
Y. T. Ching
Department of Computer Science
National Yang Ming Chiao Tung University

- Inheritance:
  - One of the most powerful OOP tools in implementing code reusability in C++.
  - New class can be created by establishing parent-child relationships with existing classes.
  - One class inherits another class's properties (attributes and behaviors).

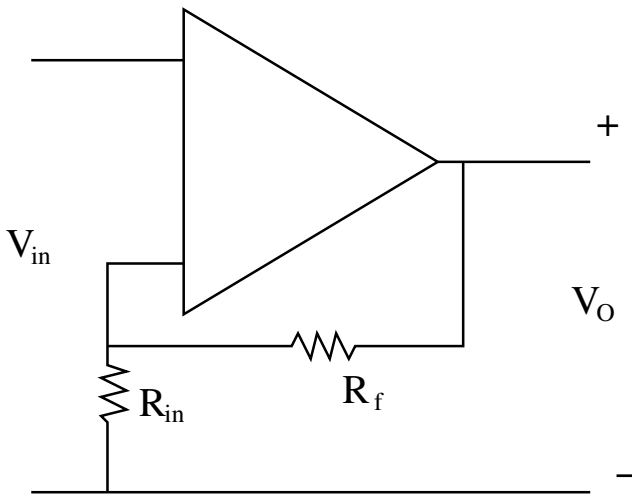
- Inheritance, called derivation, enables a hierarchy of classes to be designed.
- Hierarchy begins with the most general class and moves to more specific classes.
- In C++
  - base class- superclass
  - derived class- subclass
  - derived class inherits from the base class
  - inheritance relationship: direct and indirect.
    - vehicle, car, and coupe

- Vehicle, car, coupe
  - Vehicle: direct base class for car indirect base class for coupe
  - Car: direct derived class from vehicle direct base class for coupe
  - Coupe: direct derived class from car indirect derived class from vehicle

**C++ supports multiple inheritance**



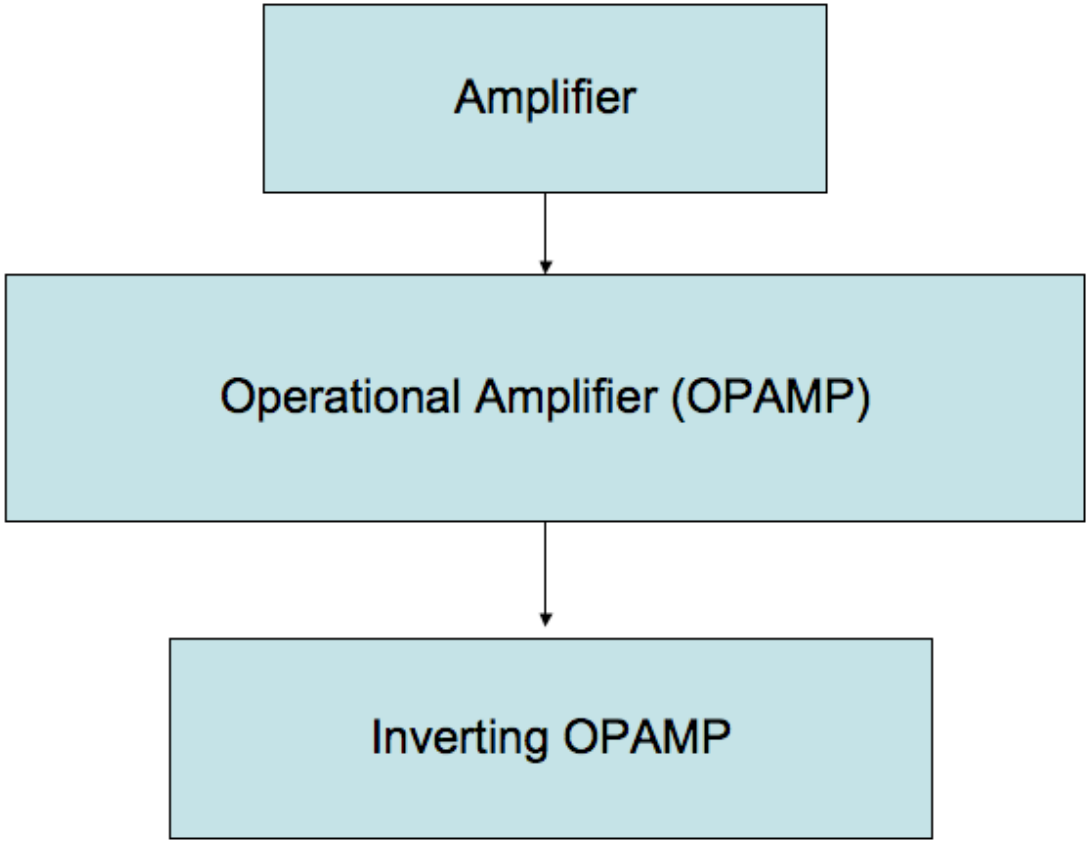
- An example
  - An inverting operational amplifier is an operational amplifier (OPAMP).
  - An OPAMP is a type of amplifier.
  - Figure 7.1




- Prog7_1, the OP Amp example. $A_v$ the voltage gain.

$$A_v = 1 + \frac{R_f}{R_{in}}$$

$$V_0 = A_v \cdot V_{in}$$


Amplifier
Operational Amplifier (OPAMP)
Inverting OPAMP

- By implementing inheritance, data members and member functions from one class (higher in the hierarchy, base class, or parent class) can become properties (members) of another class (lower in hierarchy, derived class, or child class) without coding them explicitly within this class.
- Amplifier is the base class, and OPAMP is the derived class.
- OPAMP is based class, and inverting OPAMP is derived class.
- Implementation of inheritance
  - Begin with the definition of the based class,
    - Define all properties that any derived class can inherit,
    - and some other properties that are specific to the derived class.

- General form

```
class derived_class_name:access_specifier
      base_class_name
{
  //specific properties of the derived-class
};
```

```
class Vehicle {
public:
  float mph;
  float cpmph;
  float weight;
};

class Car:  public Vehicle {
public:
  char brand_name[100];
};
```

```
class Amplifier {
public:
  float gain;
  float involt, outvolt;
  void setGain();
  void setInvolt();
  void getOutvolt();
};
```

```
class OPAMP:public Amplifier {
private:
  float rin;
  float rout;
public:
  void setRes();
  float getGain();
};
```

- The amplifier class contains six public properties.
- 3 data member `gain`, `involt`, `outvolt`.
- 3 member functions `setGain()`, `setInvolt()`, and `setOutvolt()`.
- 6 public member functions of `Amplifier` are inherited by `OPAMP` class which is derived from `Amplifier`.
- `OPAMP` contains 4 other members `rin`, `rout`, `setRes()`, and `getGain()`, totally 10 members.
- in `OPAMP`, we only have to take care `rin`, `rout`, `setRes()`, and `getGain()`.
- Reuse the codes in `Amplifier`.

## Access Specifier

- `public`: Open to the world, so the derived class can access those public members.
- `private`: Only can be accessed in the class, even the derived class cannot access those private member.
- `protected`: We should not let every member be public, so protected members are defined:
  - Used only in inheritance.
  - making all members in base class public violates the encapsulation rule, private member cannot be inherited (accessed).
  - So C++ provides an access specifier called protected.
  - To let derived class access base-class members (it sounds like public to the derived class).
  - But keep these members hidden from the rest of the world (it is still private to the rest of the world).

# `public` inheritance

- The most important access specifier
- Each `public` member in the base class is `public` in the derived class.
- Each `protected` member in the base class is `protected` in the derived class.
- Each `private` member in the base class remains `private` in the base class, so it is not visible from the derived class.

## Access Status of Derived Class's Members

| Type of Inheritance | Access Status within Base-Class | |
|---|---|---|
| | `protected` | `public` |
| | Access Status within Derived-Class | |
| `private` | `private` | `private` |
| `protected` | `protected` | `protected` |
| `public` | `protected` | `public` |

```
class Amplifier {
private:
  float involt, outvolt;
protected:
  float gain;
public:
  void setGain();
  void setInvolt();
  float getOutvolt();
};

class OPAMP : public Amplifier {
private:
  float rin;
  float rout;
public:
  void setResist();
  float getGain();
};
```

## The access specifier precedes a base-class name
## access specifier for inheritance

- defines the member's status inherited from the based class within the derived class.
  - Private access specifier: all members inherited from the base-class will be private in the derived class.
  - Public access specifier: all inherited members will have the same status in the derived class.
- In the `OPAMP` example,
  - 6 public member functions of `Amplifier` are still `public` in `OPAMP`.
  - Newly defined 4 are as they are defined in `OPAMP`.
- Default access specifier is `private`, (thus convert every inherited member to be `private`).

Members of the OPAMP class

| Member | Access Status |
|---|---|
| `rin` | private |
| `rout` | private |
| `gain` | protect |
| `setResist()` | public |
| `getGain()` | public |
| `setGain()` | public |
| `setInvole()` | public |
| `getOutvolt()` | public |

```
class Point {
  int x; // private, derived classes
  int y; // cannot see them
public:
  void set_x (int x1) {x=x1;}
  void set_y (int y1) {y=y1;}
  int get_x() {return x;}
  int get_y () {return y;}
};

class IntensePoint :  public Point {
  int intensity;
public:
  void set_intensity(int i) {intensity=i;}
  int get_intensity() {return intensity;}
};
```

- The following data members and functions cannot be inherited from a base-class
  - Private members (They are actually inherited but not accessible from the derived class).
  - Constructor and destructor functions
  - friend functions
  - static functions
  - Operator functions that overload the assignment operator

- Constructors cannot be inherited; both the base and derived classes should have their own constructors.
- When instantiating an object of the derived class, constructors of all of its parent classes are executed before the execution of the derived class constructor.
  - If `C` inherites `B` and `B` inherits `A`,
  - the order of constructor calls when instantiating an object `C` is `A-B-C`.
  - Destructors are executed in the reverse order, thus, when `C` is to be destroied, destructor executed in the order `C-B-A`.

- If a base-class constructor has arguments, these arguments also have to be added to the argument list of any class derived from this base class.
  - If `C` inherits `B` and `B` inherits `A`,
  - when instantiating an object `C`, all the arguments required by the constructors of `A, B, C` are passed to `C`.
  - `C` then passes appropriate arguments to the base-class constructors.

- A constructor initialization list must be constructed when defining the derived-class constructor.
- The general form is

```
derived_class(arg_list1):base_class(arg_list2)
{
  //body of derived-class constructor
}
```

```
class Animal {
protected:
  char species [MaxName+1];
public:
  Animal() {strcpy (species, "Animal");}
  Animal(char* s) {strcpy(species,s);}
};

class Primate:  public Animal {
  int heart_chamber;
public:
  Primate():  Animal("Primate"){};
  Primate(int n):  Animal("Primate")
    {heart_cham=n;}
};
```

```
Animal slug; //Animal()
Animal tweety("canary"); //Animal(char*)
Primate godzilla; //Primate()
Primate human(4); //Primate(int)
```

**Each Primate's constructor invoke Animal's constructor first**

If a base class has a constructor but no default constructor, then a derived class constructor must explicitly invoke some base class constructor.

```
class B {
  int x;
  int y;
public:
  B(int a) {x=a; y=999;}
  B(int a1, int a2)
    {x=a1; y=a2;}
};
class D: public B{
  int z;
public:
  D(int n) {z=n;}
  // ERROR must invoke one of the two constructors
  // OR B has a default constructor
};
```

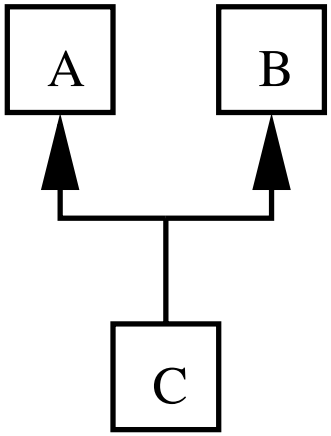
```
class B {
  int x;
public:
B() {cout << "B::B() fires " << endl;}
};
class D{
  int x;
public:
  D() {cout << "D::D() fires "<< endl;}
};
int main()
{ D d; }

B::B() fires
D::D() fires

D: B() {...} //legal but not necessary
```

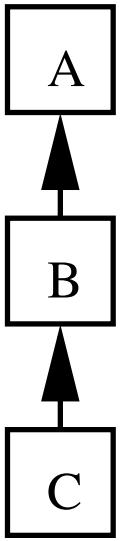
## Rules

`D` a class derived from `B`

- `D` has constructors but `B` has not, then the appropriate `D` constructor fired whenever a `D` object is created.
- `D` has no constructors but `B` has, `B` must have a default constructor.
- `D` has constructors, `B` has a default constructor, `B`'s default constructor invoked if a constructor of `D` did not invoked a `B`'s constructor.
- Both `B` and `D` have constructor. `B` has no default. `D`'s constructor must invoke a `B`'s constructor.

- Single inheritance, a derived class has only one parent class.
- There is multiple inheritance,
  - Direct multiple inheritance: derived class can directly inherit more than one base-class.
  - Indirect multiple inheritance.



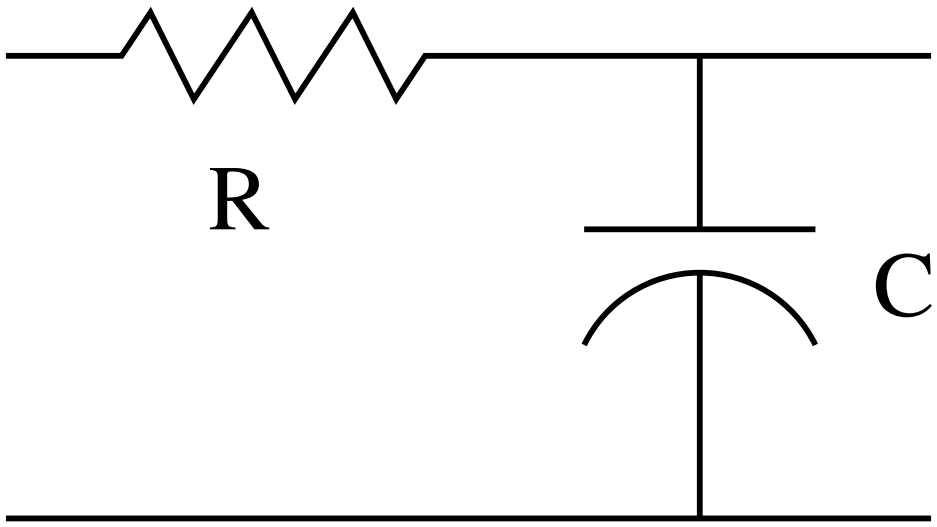
```
class Derived_class:access_specifier Base1_class,
      access_specifier Base2_class, ...,
      access_specifier BaseN_class
{
//body of derived class
}
```

# Indirect Inheritance

```
class Animal {
protected:
  char speciesInLatin[100];
  float lifeExpectancy;
  int warmBlood_P;
};

class Cat:  public Animal {
protected:
  char range[100][100];
  float favoritePrey[100][100];
};
```

```
Class HouseCat:  public Cat {
  char toys[10000][100];
  char catPsychiatrist[50];
  char catDentist[50];
  char catDoctor[50];
  char apparentOwner[50];
};
```

- A low-pass filter: a circuit that is designed to pass signals with low frequencies and reject signals with high frequencies.
- A Typical low-pass filter is designed from an RC circuit.
- $\omega_c = 1/RC$ is the cut off frequency,
- Problem: Given $R = 1000\Omega$, $C = 4.7 \cdot 10^{-7}$F, determine $\omega_c$
- Solution: $\omega_c = 1/RC = 2127.66$ rad/s.



# RC Circuit

```
#include <iostream>
using namespace std;
class RC_Circuit {
protected:
  double res;
  double cap;
public:
  RC_Circuit(double r, double c) {res=r;cap=c;}
  void setRes() {cout<<"\tEnter R = ";cin>>res;}
  double getRes() {return res;}
  void setCap() {cout<<"\tEnter C = "; cin>>cap;}
  double getCap() {return cap;}
};
```

```
class Filter:public RC_Circuit {
protected:
  double frq;
public:
  Filter (double r, double c, double f):RC_Circuit(r,c)
    {frq=f;}
  double getFreq() {return frq;}
};

class Low_Pass:public Filter {
  double frq_cut;
public:
  Low_Pass (double, double, double);
  void setCutoff() {frq_cut=1/(res*cap);}
  double getCutoff() {return frq_cut;}
  bool isPreserved();
};
```

```
Low_Pass::Low_Pass(double r, double c,
    double f):Filter(r,c,f)
{
  setCutoff();
}

bool Low_Pass::isPreserved()
{
  return ((frq<=frq_cut)?  true :  false);
}
```

```
int main(int argc, char * argv[])
{
  Low_Pass filter(1000, 0.00000047, 2500);
  cout<<"Low-Pass Filter:"<<endl;
  cout<<"\tR= "<<filter.getRes()<<" ohms"<<endl;
  cout<<"\tC= "<<filter.getCap()<<" F"<<endl;
  cout<<"\tCutoff Frequency = "<<filter.getCutoff()
    <<" rad/s";
  if (filter.isPreserved()) {
    cout<<"\n\tFrequency = "<<filter.getFreq();
    cout<<" rad/s is preserved." <<endl;
  }
  else {
    cout<<"\n\tFrequency = "<<filter.getFreq();
    cout<<"rad/s is attenuated."<<endl;
  }
```

```
  cout<<"\n\nAfter changing R and C:"<<endl;
  filter.setRes();
  filter.setCap();
  filter.setCutoff();
  cout<<"\tCutoff Frequency = "<<filter.getCutoff()
    <<" rad/s";
  if (filter.isPreserved()) {
    cout<<"\n\tFrequency = "<<filter.getFreq();
    cout<<" rad/s is preserved." <<endl;
  }
  else {
    cout<<"\n\tFrequency = "<<filter.getFreq();
    cout<<" rad/s is attenuated."<<endl;
  }
  return 0;
}
```

- Constructors are not inherited.
- `public` access specifier so
  - private member: `frq_cut`.
  - protected members: `res`, `cap`
  - public members: `Low_Pass()`, `setFreq()`, `setRes()`, `getRes()`, `setCap()`, and `getCap()`.

- `Low_Pass` is derived from `Filter`, `Filter` is derived from `RC_Circuit`
- `Low_Pass` inherited directly from `Filter`, and indirectly from `RC_Circuit`.
- Constructors are executed in the order from `RC_Circuit` to `Filter`, then to `Low_Pass`.
- Destructors are executed in the reverse order.

## Dominating and Overriding Base Class Members

- In multiple inheritance, what if variables are defined in the same name in the base and derived classes?
- In general, variables (functions) defined in the derived class overrides variables (functions) defined in the base class.
- to access members inherited from a base class by using a derived class object
- `derived_object.base_class_name:: inherited_member;`

```
class B {
public:
  int x;
};

Class D: public B {
public:
  int x;
};

int main()
{
  D d1;
  d1.x = 999; // D::x, not B::x
  d1.B::x = 4; //ok
}
```

- Prog 7_4, this program aims to show some important aspects of inheritance, NOT to illustrate common programming practices.

```
#include <iostream>
using namespace std;
class Resistor {
public:
  float r; //both Resistor and Circuit have r
  Resistor(float x=0){r=x;}
  void getVal() {cout<<"R = "<<r<<endl;}
    //both Risistor and Circuit have getVal()
};

class Circuit:public Resistor {
public:
  float r; //both Resistor and Circuit have r
  double c;
  Circuit(float x, double y):Resistor(330) {r=x;c=y;}
  void getVal() {cout<<"R = "<<r<<" C = "<<c<<endl;}
    //both Risistor and Circuit have getVal()
};
```

```
int main(int argc, char* argv[])
 {
  Resistor res1(470);
  Circuit cir1(200,0.0001), cir2(1000, 0.0005);
  cout<<"\t\tAccessing data members:\n\n";
  cout<<"res1.r = "<<res1.r<<endl;
  cout<<"cir1.r = "<<cir1.r<<endl;
  cout<<"cir2.r = "<<cir2.r<<endl;
  cout<<"cir1.Resistor::r = "<<cir1.Resistor::r<<endl;
  cout<<"cir2.Resistor::r = "<<cir2.Resistor::r<<endl;
```

```
  Resistor res2=cir2; //
  cout<<"res2.r = "<<res2.r<<endl;
  cout<<"\n\t\tExecuting member functions:\n\n";
  res1.getVal();
  cir1.getVal();
  cir1.Resistor::getVal();
  cout<<"\n\t\tUsing a base-class pointer:\n\n";
  Resistor *rptr;
  rptr=&res1;
  rptr->getVal();
  rptr=&cir1;
  rptr->getVal();
  return 0;
}
```

- A pointer to the base class can point to the derived class but can only access those inherited members.
- A pointer to the derived class cannot point to a base class but can be cast to point to a based class.

| | Object type | Object type |
|---|---|---|
| Pointer type | Base-class | Derived-class |
| Base-class | No Problem | Error |
| Derived-class | No Problem | No Problem |

## Sample Application: Measuring Computer Performance

- 3 inheritance hierarchies, base classes are Bmark, Computer, and Test.
  - `Computer` has `Desktop`, `Mainframe`
    - Desktop has WS, PC
  - `Bmark` has `WetRock`, `DryRock`, and `PetRock`.
  - `Test` consists of a computer running a `Bmark`.
  - `Test` is a friend to computer and `Bmark`

```
#include <iostream.h>
#include <string.h>
#include <math.h>
const int MaxName = 100;
const float Tolerance = 0.01;

class Test;
class Bmark {
friend Test;
protected
  float alP; //arithmetic-logic
  float mP; // memory
  float cP; // control
  float ioP; // io
  float tlP; // tight-loop
  float ic;/ executed instruction count
  char name[MaxName+1];
public:
```

```
// in the public section of BMark
BMark()
{
  init();
  strcpy (name, "???");
}

Bmake (char* n)
{
  init();
  if (strlen(n) < MaxName)
    strcpy(name,n);
  else
    strcpy (name, n, MaxName);
}
```

```
Void report()
{
  cout << "Benchmark "<< name << endl;
  cout << "Total instructions executed =="
    << ic << endl;
  cout << "Arithemetic-logic =="
    << alP << endl;
  cout << "Memory == " << mP << endl;
  cout << "Control == " << cP << endl;
  cout << "Input/Output == " << ioP << endl;
}

int ok()
{
  return fabs(1.0-(alP+mP+cP+ioP))
    <= Tolerance;
}
```

```
  void init_error()
  {
    cout << name << "-- init error!" <endl;
  }
private:
  void init()
  {
    alP=cP=mP=ioP=ic=0.0;
  }
};
```

**End of class BMark definition.**

```
// benchmark emphasizes arith-logic control statement
// with moderate traffic and low io

class WetRock():  public Bmark {
public:
  WetRock():  BMark("WetRock")
  {
    alP = 0.5;
    cP = 0.2;
    mP = 0.2;
    ioP = 0.1;
    ic = (float) 4500301;
    if (!ok()) init_error();
  }
};
```

```
// benchmark emphasizes arith-logic control statement
// with light traffic and no io

class DryRock():  public BMark {
public:
  DryRock():  BMark("DryRock")
  {
    alp = 0.77;
    cp = 0.166;
    iop = 0.0;
    mP = 1.0 - alP-cP-ioP;
    ic = (float) 6700909;
    if (!ok()) init_error();
  }
};
```

```
// benchmark emphasizes memory traffic with moderate
// I/o and low arith-logic and control

class PetRock():  public BMark {
public:
  PetRock():  BMark("PetRock")
  {
    alp = 0.122;
    cp = 0.577;
    iop = 0.239;
    mP = 1.0 - alP-cP-ioP;
    ic = (float) 10400500;
    if (!ok()) init_error();
  }
};
```

```
class Computer {
friend Test;
protected:
  // cpi = cycles per instruction
  float alcpi, ccpi, iocpi, mcpi;
  float ct; //cycle time
  char name[MaxName+1];
  float costU, CostL;
protected:
  Computer (float al, float c, float io, float m,
    float t, char* n, float l, float u)
  {
    alcpi = al; ccpi = c; iocpi = io;
    mcpi = m; ct = t;
    if (strlen(n)<MaxName) strcpy(name,n);
    else strcpy (name,n,MaxName);
    costU = u; costL = l;
  }
```

```
// in class Computer
  void report()
  {
    cout << "Computer " << name
      << "at cost ranging from $" << costL
      << "to $" << costU << "." endl;
    cout << " Clock cycle (nanoseconds):"
      << ct << endl;
    cout << " CPIs:  " << emdl;
    cout << "Arith-Logic:  " << alcpi << endl;
    cout << "Control:  " << ccpi << endl;
    cout << "Memory:  " << mcpi << endl;
    cout << "I/O " << iocpi << endl;
  }
//end of class Computer
};
```

```
// desktop machine comprise workstations and
// personal computer

class Desktop:  public Computer {
protected:
  Desktop ( float al,
    float c;
    float m;
    float io;
    float t;
    char* n;
    float l;
    float u;)
    :  Computer:(al, c, m, io, t, n, l, u){}
};
```

```
Class PC: public Desktop {
public:
  PC( float al=1.8, float c=2.3, float m=5.6,
    float io=9.2, float t=230, char* n="PC",
    float l=800.00, float u=14500.00 )
      :  Desktop (al,c,m,io,t,n,l,u) {}
};

Class WS: public Desktop {
public:
  WS( float al=1.3, float c=1.7, float m=2.1,
    float io=5.8, float t=15, char* n="WS",
    float l=4500.00, float u=78900.00 )
      :  Desktop (al,c,m,io,t,n,l,u) {}
};
```

```
class Mainframe:  public Computer {
public:
  Mainframe(float al=1.2,
    float c=1.5,
    float m=3.6,
    float io=3.2,
    float t=50,
    char* n="$$",
    float l=310000.00,
    float u=20000000.00 )
      :  Computer (al,c,m,io,t,n,l,u) {}
};
```

```
class Test {
  float rt; // response time
  void result (Computer c, Bmark b);
public:
  Test (Computer c, Bmark b);
};
```

```
Int Test::Test (Computer c, BMark b)
{
  float al_rt, c_rt, m_rt, io_rt;

  al_rt = b.alP*b.ic*c.alcpi*c.ct;
  c_rt = b.cp*b.ic*c.ccp*c.ct;
  m_rt = b.mP*b.ic*c.mcpi*c.ct;
  io_rt= b.ioP*b.ic*c.iocpi*c.ct;
  rt = al_rt+c_rt+m_rt+io_rt;
  result(c,b);
}
```

```
Void Test::result(Computer c, Bmark b)
{
  cout << "\nBenchmark result:\n" << endl;
  cout << " RT (ns) for " << b.name;
    << "=" << rt << endl;
  cout << "\n Benchmark and computer
    details follow.\n"
    << endl;
  b.report();
  c.report();
}
```